VDLM · VIENNA · 2026

Architecting Agentic AI for Production.

SPEAKERS

Hilda Kosorus · Co-founder & AI Engineer

Tobias Rechberger · Co-founder & AI Engineer

Csenge Szabo · AI Engineer

onefold.ai

Hilda Kosorus

Tobias Rechberger

Csenge Szabo

Agentic AI.
In production.

Vienna-born. Engineering-driven.
We design, build and deploy production-grade agentic systems — end to end.

25+
years combined experience

50+
AI engagements delivered

E2E
strategy to production

100%
European team

MOTIVATION

Demo day vs. production day

THE DEMO

2 days.

- Happy path only — one user, one task, curated/simple data
- No retries, no partial failures, no cost concerns
- No observability needed
- Impressive, but fragile - and it's ok

PRODUCTION

6 months.

- Real users, real edge cases, real infrastructure constraints
- State, memory, tool failures, model drift, prompt drift
- You need to know what the agent did — and why
- This is what we are going to talk about today.

This is hard!

SEVEN ASPECTS TO CONSIDER

The gap isn't one problem

01
Infrastructure
Compute, latency budgets, async execution, containerisation.

02
Deployments
Versioning, rollbacks, zero-downtime prompt changes.

03
Data access & quality
Retrieval freshness, knowledge graph consistency, schema drift.

04
UX
Latency expectations, streaming, progressive disclosure of agent reasoning.

05
Domain understanding
Regulatory context, vocabulary, edge cases that only surface in the real domain.

06
State & memory
What each agent knows, what it forgets, what carries across sessions.

07
Tool selection
Which tool, when, at what cost — and what happens when it fails.

Plus
The ecosystem
Too many frameworks, too many patterns, too easy to thrash.

WHERE THESE LESSONS COME FROM

Examples of agentic systems we delivered

COMPLIANCE · AUDIT

AI Audit Assistant

Multi-agent and RAG-based system for assisting auditors in their workflow: preparation, execution & documentation.

Challenge: Complex and varied data sources with accurate citations. Accuracy and clarity is key.

FOOD SCIENCE · R&D

Food Science Workspace

Agentic workspace for recipe formulation and optimization integrating supplier data, scientific literature and proprietary formulation guidelines.

Challenge: Agent-tool orchestration and navigating the proprietary knowledge graph.

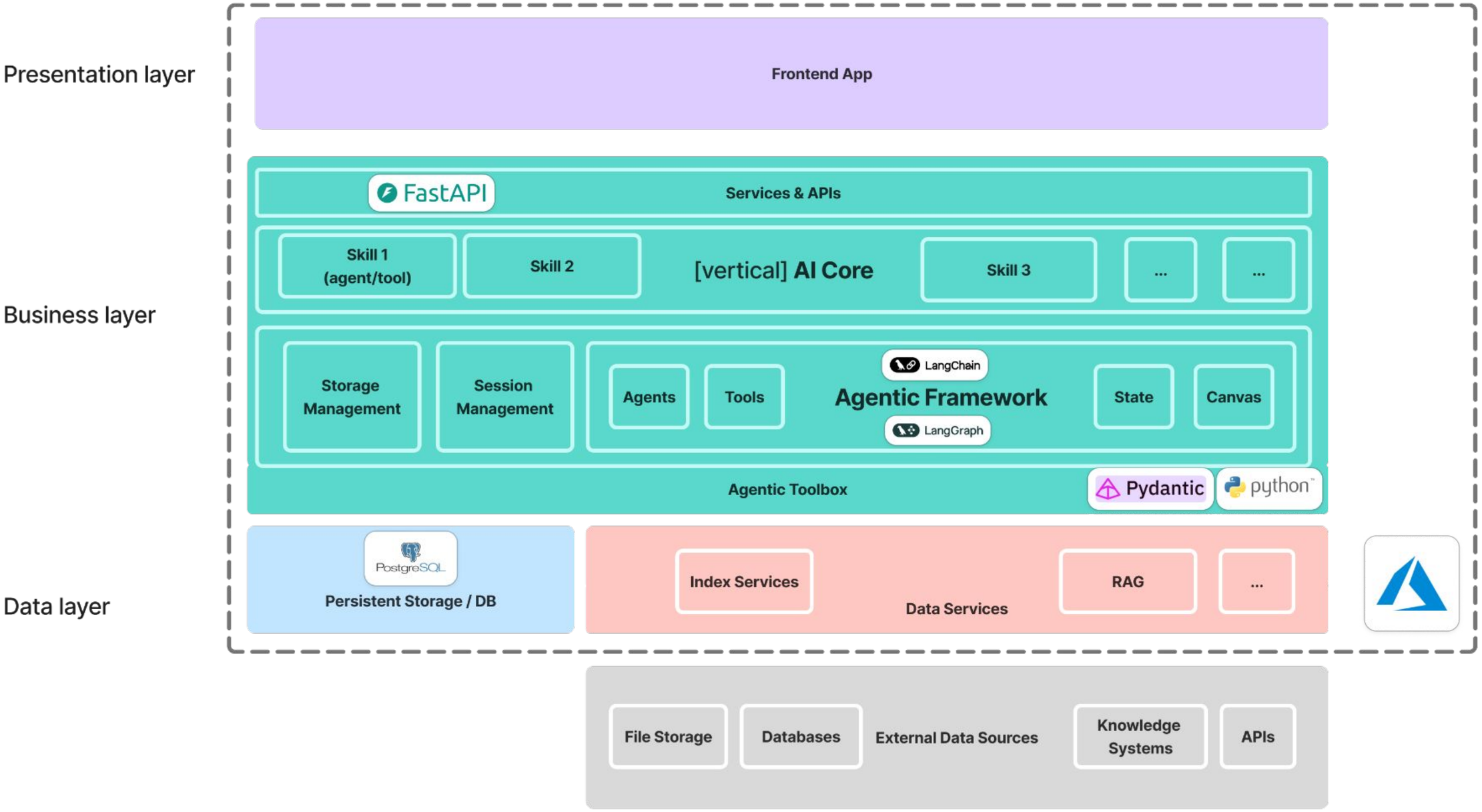
INTERNAL · ONEFOLD

Our Own Product

Agentic platform accelerator that enable fast vertical AI application deployments, including our own vertical solution for AI strategy development.

Challenge: Keeping up with the fast paced tech developments, tech stack choice + new skill development.

Our agentic architecture today.


Presentation layer
Frontend App
Business layer
FastAPI
Services & APIs
Skill 1 (agent/tool)
Skill 2
[vertical] AI Core
Skill 3
...
...
Storage Management
Session Management
Agents
Tools
LangChain
Agentic Framework
LangGraph
State
Canvas
Agentic Toolbox
Pydantic
python
Data layer
PostgreSQL
Persistent Storage / DB
Index Services
Data Services
RAG
...
File Storage
Databases
External Data Sources
Knowledge Systems
APIs

THE STACK

Agentic components

Orchestration layer
LangGraph supervisor graph — coordinates sub-agents, manages state, handles retries and handoffs.

LangChain
LangGraph

RAG & knowledge layer
Vector stores + knowledge graphs. Retrieval quality is a first-class concern, not an afterthought.

PostgreSQL + pgvector

Tool layer
Typed tool registry with failure handling. Every tool is observable, every call is logged.

LangChain

Observability layer
Langfuse for tracing, evals and prompt management across all agents. Full trace on every request.

Langfuse

Orchestration

Coordinating multiple agents, state, memory, tool routing and recovery from failures — this is where the hard problems live.

ORCHESTRATION

Orchestration is the hard part.

A single LLM call is a solved problem.

Chain two agents and you have a distributed system.

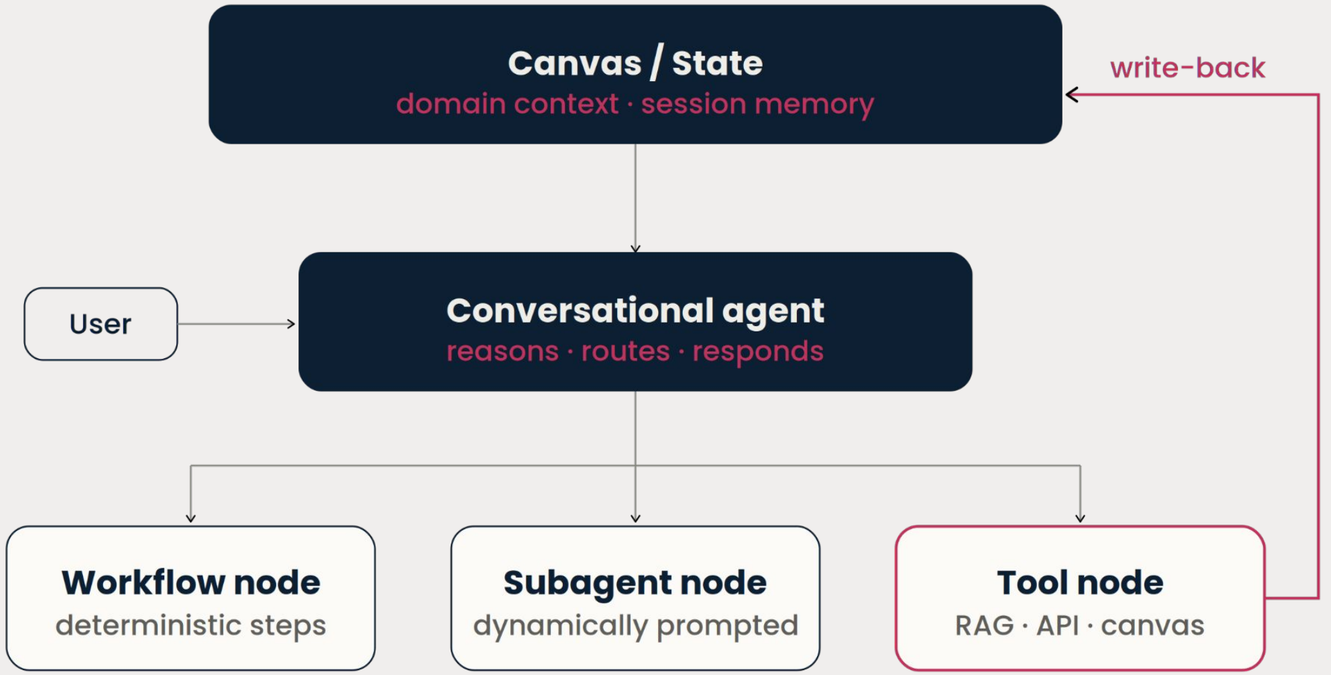
With all the failure modes that come with it, plus new ones unique to language models.

- **Coordination** — who decides what happens next, when, and based on what context
- **State** — what each agent knows at each step, and what it doesn't
- **Memory** — short-term context vs. long-term knowledge, and when to use which
- **Tool routing** — which tool, how many times, at what cost
- **Recovery** — retries, fallbacks, graceful degradation when a sub-agent fails

EXAMPLE

What orchestration looks like in practice.

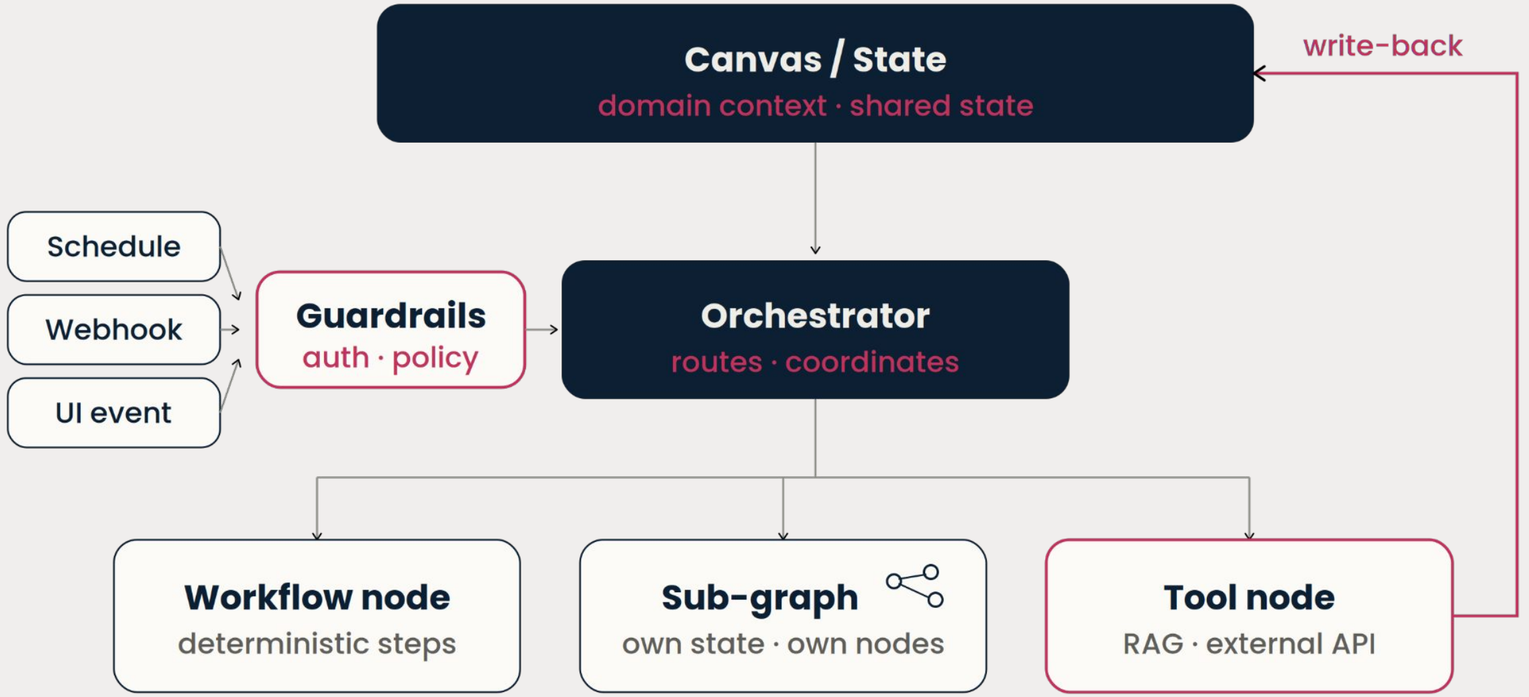
We use LangGraph: Explicit graph structure, typed state, auditable routing. Example:



EXAMPLE

What orchestration looks like in practice.

We use LangGraph: Explicit graph structure, typed state, auditable routing. Example:



Patterns we converged on.

PATTERN 01

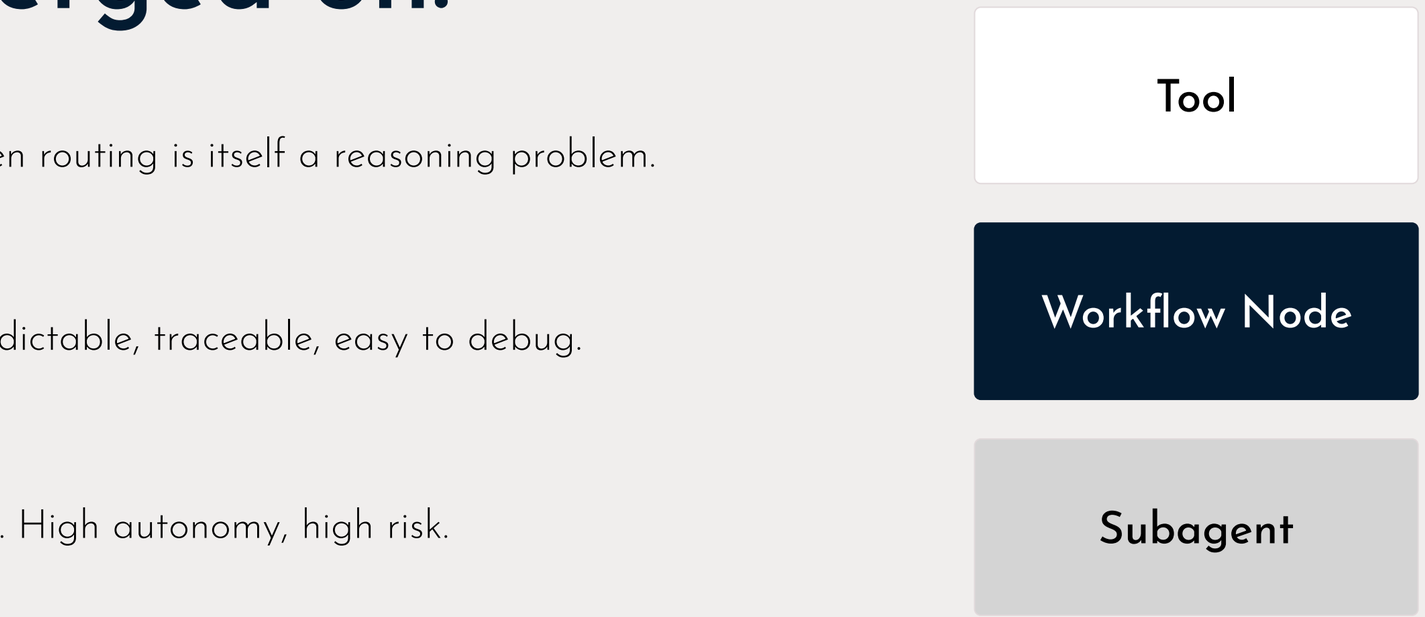
Supervisor one LLM orchestrates. Good when routing is itself a reasoning problem.

PATTERN 02

Handoff agents pass control explicitly. Predictable, traceable, easy to debug.

PATTERN 03

Swarm loosely coupled, self-coordinating. High autonomy, high risk.



LESSONS LEARNED

Where We Pulled Back.

The framework decision

We built custom agents.

- No mature framework available at build time
- We owned routing, tools, agents, state, history, retries, ...
- LangGraph stabilized with v1, we embraced the ecosystem
- There are still cases where owning the abstractions is valid, but maintaining boilerplate comes at a cost

Agent autonomy

We trusted agents too much.

- Partial failures looked like success
- Non-deterministic behaviour we couldn't reproduce
- Cost accumulation from agent retries
- Pulled back: Not because autonomy is wrong, but because unobservable autonomy is

THE PAYOFF

Tailoring is how you get all three.

Quality - a horizontal app does its best.

A tailored system is grounded in domain data, constrained tools, structured outputs. The model has less room to hallucinate and more signal to work with.

Safety - a generic system has no baseline for correct.

When you know what the agent should and shouldn't do, you can enforce it. Constrained tools, bounded scope, structured outputs, each one narrows the failure surface.

Observability - logs without a reference point are just noise.

When you know what correct looks like, deviations become visible and failures become meaningful. Tailoring is what gives you that baseline to measure against.

Evaluation & Observability.

How to find out whether your system behaves correctly — and how to build the loop that catches it when it doesn't.

RAG Eval ≠ Agentic RAG Eval

Metrics for RAG Evaluation

Faithfulness

Is every claim in the answer grounded in the retrieved context?

Response Relevancy

Does the answer actually address the question asked?

Context Precision

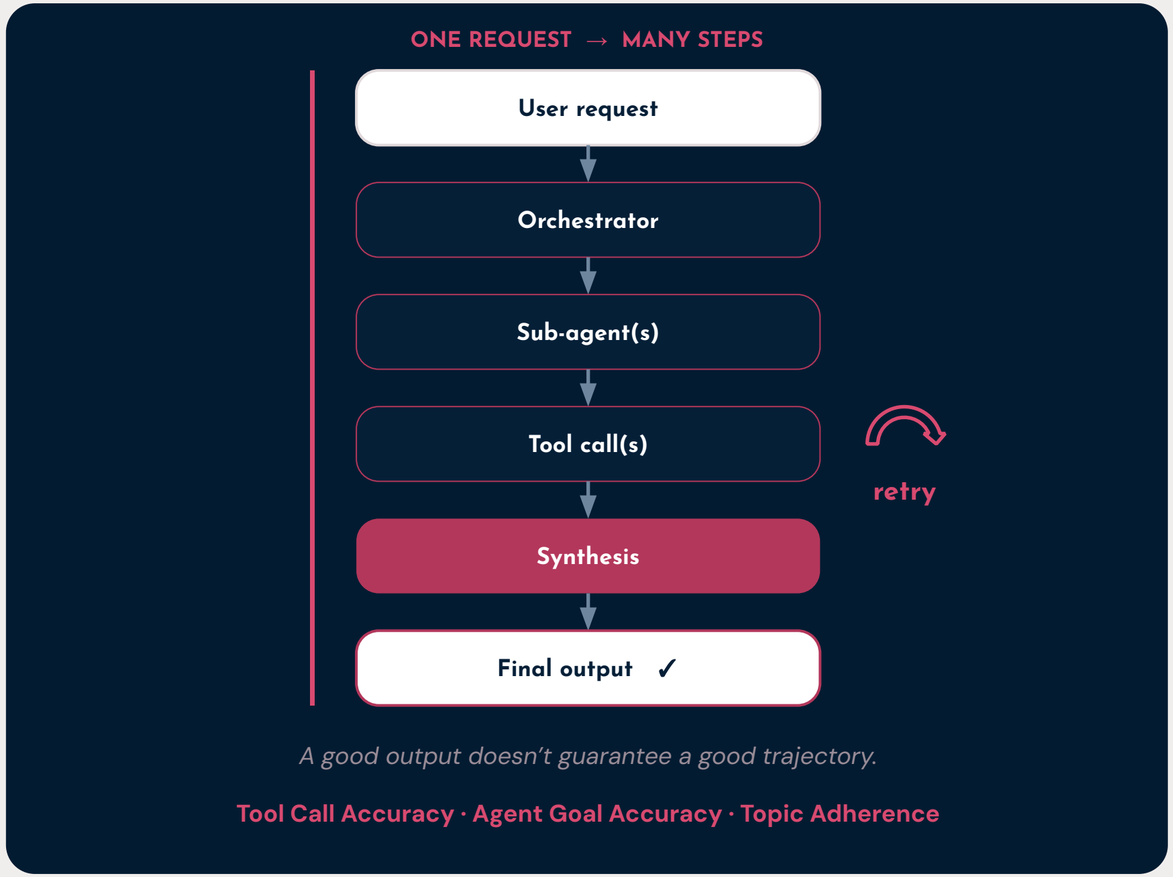
Are the retrieved chunks actually relevant to the question?

Context Recall

Did retrieval pull in everything needed to answer well?

These work because RAG is a single trace: query → retrieval → generation.
The path is fixed and the context is known, so each metric checks a single step against a single output.

docs.ragas.io

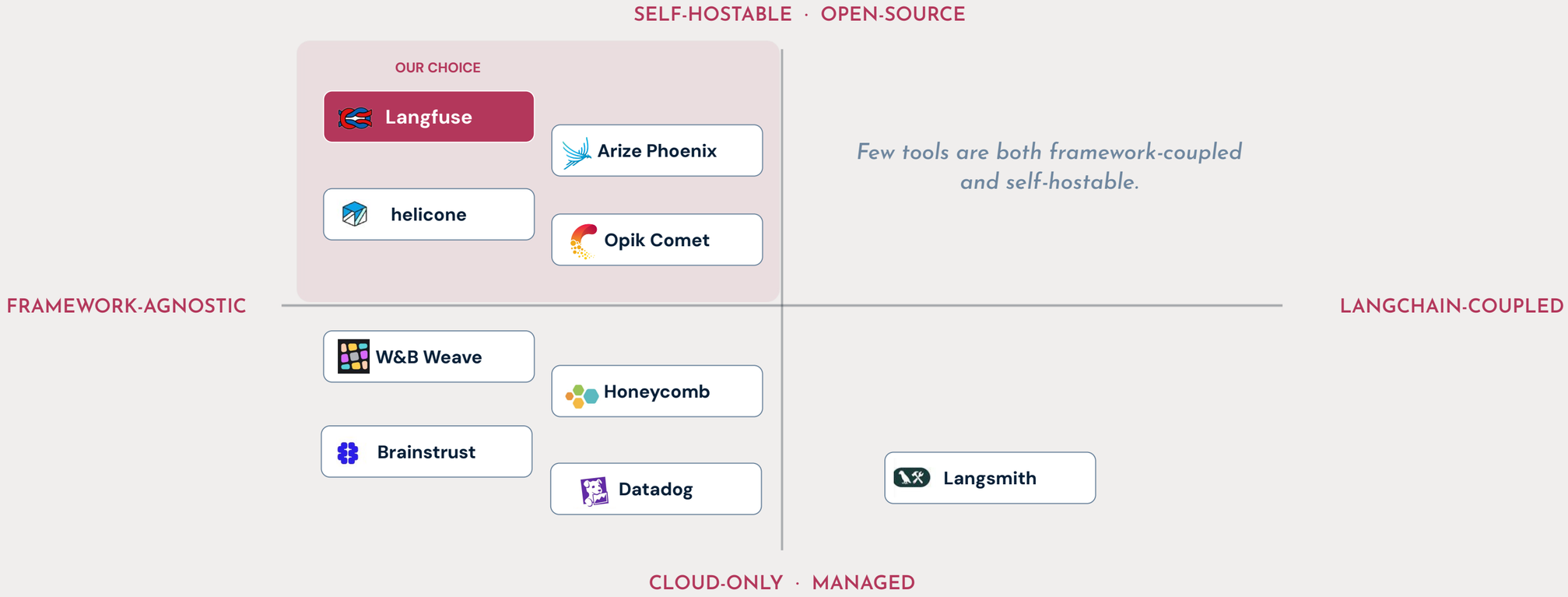


We're not only scoring the output. We're scoring the entire **trajectory.**

OBSERVABILITY LANDSCAPE

Two questions narrow the field

How coupled to a framework — and how willing are you to self-host? The whole field sits on those two axes.



WHY WE CHOSE LANGFUSE

Langfuse earned its place

Why we trust it

Open-source (MIT)
We can read the code, and never get cornered on pricing.

Self-hostable
Runs on our own infrastructure — Docker, Kubernetes or Terraform, on any cloud.

EU & US data regions
Our data stays in-region, which our compliance work requires.

SOC 2 Type II · ISO 27001 · GDPR · HIPAA-eligible
Security certifications come built in.

Open Telemetry-native
Swap frameworks if we want and the traces stay the same.

What it can be used for

Hierarchical tracing
Zoom from a whole session down to one span, with cost and latency on each.

Versioned prompt management
Every change is tracked, so we can compare versions, see the diff, and roll back.

LLM-as-judge, heuristic & human evals
All three scoring methods in one place, on the same data.

Datasets & experiments
Turn traces into a test set and run experiments on it.

Annotation queues & playground
Human-in-the-loop review and interactive prompt iteration in the same tool.

langfuse.com/docs

OPEN-SOURCE EVAL FRAMEWORK

RAGAS — How the field caught up

ABOUT RAGAS

LLM-as-judge & rubrics

- Open-source Python library for evaluating LLM-based applications.
- Scores with LLM-as-judge against structured rubrics.
- Every metric is a 0–1 score you can track over time.

USE CASE 1

Synthetic test data generation

- Builds a knowledge graph from your documents.
- Generates test queries from *scenarios* and *personas*.
- Useful for running evals without having any prod data.
- Human-in-the-loop makes the data always richer.

USE CASE 2

Eval metrics for RAG

Faithfulness · Response Relevancy · Context Precision · Context Recall

Eval metrics for Agents

Topic Adherence
Did the agent stay on predefined domains during the interaction?

Tool Call Accuracy
Did the agent invoke the right tools and right arguments in the right order?

Agent Goal Accuracy
Did the agent identify and achieve what the user actually wanted?

docs.ragas.io

→ **Every RAGAS score pushes straight into Langfuse, attached to the trace it came from.**

THE EVAL FLYWHEEL

Production traces are your long-run signal

Offline & Online — offline curated datasets pre-deploy to catch drift before users do. Online on sampled traffic continuously. You need both.

Production traces → Annotation Queue — every run is a candidate. Flag surprising, bad or interesting traces for human review.

Annotated traces → Curated Dataset — reviewed examples become ground truth. Every prod failure is next week's regression test.

CI/CD — Langfuse's GitHub Action runs the evals on every pull request. If scores drop below threshold → build fails automatically.

WHAT OBSERVABILITY CATCHES

What breaks your system in production

01 Prompt drift

Someone tweaks a system prompt to fix one thing. An unrelated downstream behavior breaks a week later — no one connects the two.

→ Version every prompt. Run every change through your eval suite before it ships.

02 Silent tool failures

Retries swallow errors, partial successes report as successes, and the agent makes something up. Nobody notices until a user reports the bug.

→ Instrument the tool layer. Track success rate per tool, not per request.

03 Model drift

Provider ships a minor version, your eval scores tick down 2%, and nothing alerts. Without a regression suite on a pinned dataset, you'd never notice.

→ Pin model versions. Re-run your evals when you swap — minor bumps included.

04 Cost regressions

A tier change may double inference cost. Staying on deprecated model versions instead of swapping to a newer one can cost about 3× more.

→ Track cost per agent, per span — not just total. Observability is how you find it.

ESTABLISHING TRUST

Observability isn't optional

WITHOUT IT

- × Users find your bugs first
- × Inference cost balloons silently
- × Every incident is forensic archaeology
- × You ship slower, afraid of what might break
- × The same failure keeps recurring

WITH EVAL + OBSERVABILITY

- → You catch regressions in CI, before users do
- → Cost is attributable per span, agent, and user
- → You debug from the trace, in minutes
- → You ship faster, because you can debug easily
- → Every failure becomes next week's regression test

Observability is how you ship agents you can trust.

Some Takeaways

WHAT IT TAKES

Three practices required for production

1. PRACTICE

Orchestrate deliberately

Use deterministic paths where the flow is known. Reserve agents for tasks that genuinely need flexibility. When you do, explicit graph topologies, typed state, and structured handoffs keep them in control.

2. PRACTICE

Observe properly

Tracing, cost attribution, and eval datasets aren't a phase-two problem. Build observability in from the start — not as an afterthought once things break.

3. PRACTICE

Test thoroughly

Capture failures in production. Turn them into test cases. Run experiments before you ship. Most teams intend to do this. Few actually build the habit.

ONEFOLD AI

WE'RE HIRING

AI Engineer

Vienna • Hybrid • Full-time • IT KV, adjusted to experience • English + (bonus) German-speaking

What you'll build

- Agentic workflows — orchestration, tool use, multi-step reasoning
- RAG pipelines end to end — chunking, embeddings, retrieval, re-ranking
- Evaluation pipelines — ship, measure, iterate on data
- Messy real-world data — PDFs, scans, unstructured enterprise content

What we offer

- Company participation (UWAs) — you're invested in what you build
- Real autonomy paired with real accountability
- Learning budget for conferences, courses, staying current
- Hybrid Vienna setup — no performative office time

careers@onefold.ai

Discussion & Questions

ONEFOLD AI

Thank you.

Website

onefold.ai

LinkedIn

linkedin.com/company/onefold-ai